

# COMP6841 Week 1 Portfolio

Name: Thomas Gosman

zID: z5425064

Date Submitted:

## 1. Setup

### Security Everywhere

Coming from an embedded and hardware background, I found this week's discussion on AI and security theatre very interesting. We often focus on visible AI risks, such as whether a model is training on our data, but we also need to consider what happens to that data while it is being encrypted, stored, and processed.

ON THIS PAGE

[Summary](#)

[Affected Products and Mitigation](#)

[Acknowledgement](#)

[References](#)

[Revisions](#)

[Back to Security Bulletins and Briefs](#) ↩

### Undervoltage-based Static Side-channel Attacks ("Chypnosis") on FPGAs

AMD ID: AMD-SB-8018

**Potential Impact:** Loss of Confidentiality

**Severity:** Medium

#### Summary

This document describes a potential attack technique against FPGA devices that leverages side-channel analysis (SCA) techniques to physically extract register and memory content from the device. In applications following best practices for security, critical data, such as decryption keys, is only retained in device registers and memories for a limited time. The newly reported attack makes use of a brownout condition to put the device into a hibernation state in which the transistors cannot switch anymore, but the register and memory content is preserved<sup>1</sup>. To achieve this, the attacker rapidly lowers the supply voltage below the logic operation threshold but above the data retention threshold. On-chip mechanisms to detect the voltage drop and react by clearing critical data are rendered ineffective by the fast voltage drop. Once the hibernation region has been entered, the attacker has unlimited time to extract the data.

The academic research paper introducing the new approach demonstrates the attack on the programmable logic (PL) in AMD Artix™ 7-Series FPGA devices. It shows that the on-chip XADC-based voltage monitor is too slow to detect and/or execute a tamper response to clear memory contents. Furthermore, they show that detection circuits that have been developed to detect clock freezing<sup>2</sup> are ineffective as well. In general, the attack can be applied on all ICs that do not have effective tamper responses to clear sensitive data in case of an undervoltage event.

Different cloud service providers may use FPGAs for specialised tasks such as encryption, networking, and acceleration, while FPGAs are also widely used in edge AI deployments where low latency, reliability, and high performance are critical. An FPGA, (Field-Programmable Gate Array), is a silicon chip whose digital circuitry can be reconfigured after manufacturing, almost like a configurable processor that can be customised for a specific task.

This flexibility makes FPGAs especially valuable in security-sensitive and real-time systems where speed and determinism matter, such as datacentre encryption, packet filtering, load balancing, high-speed routers, medical imaging, air traffic control systems, video processing, and autonomous vehicle sensors. I think of them as hardware specialists: instead of being general-purpose processors, they can be shaped to perform one important job extremely fast, reliably, and efficiently.

This article highlighted that in certain powering conditions, particularly where attackers have physical access or privileged control over hardware, even strong encryption can be weakened through side-channel attacks. These attacks do not break the encryption algorithm itself, but instead exploit a voltage change to freeze the chip from execution but leaving readouts from memory and flip flops available. By putting the processor into this state allows the control plane to be moved into the data plane, which can have huge ramifications.

For AI systems, this matters most in high-value infrastructure such as cloud datacentres, edge AI devices, defence systems, and secure accelerators. In these cases, an attacker may target encrypted user data, secret cryptographic keys, or even the intellectual property implemented within the FPGA itself. In many cases, this IP can represent years of engineering effort and be worth millions of dollars as a closely guarded trade secret.

This was a fascinating example of an attack surface that many developers may not initially consider. It reinforced the idea that security engineering is rarely about achieving perfect security. Instead, it is about identifying threats, reducing risk, and ensuring that systems can respond and recover when vulnerabilities are inevitably discovered.

Source Here: <https://www.amd.com/en/resources/product-security/bulletin/amd-sb-8018.html>

PS: I thought a photo of the title is would be pretty boring as an image. So to spice things up I got AI to go on a bit of creative journey and try to make as many references about LOTR in the context of chynposis as possible, it is a surprisingly good summary if you want a quick idea about what this is. (Also their are lots of errors details in this which can be a reminder not to trust AI alone) [1]



# CHYPNOSIS

ONE GLITCH TO RULE THEM ALL

IN MIDDLE-EARTH, DARK POWERS SEEK:

- LEAKED DATA
- EXPOSED KEYS
- STOLEN IP
- MILLION-DOLLAR RESPINS
- CLASSIFIED SECRETS
- ... AND MORE



A WISE FPGA DOES NOT TRUST IN ITS OWN SHIELD.

## CHYPNOSIS

Undervoltage-Based Side-Channel Attacks on FPGAs



MORDOR

ONE RING TO RULE THEM ALL



EVEN THE STRONGEST ENCRYPTION CAN BE CORRUPTED.

PRECIOUS SECRETS



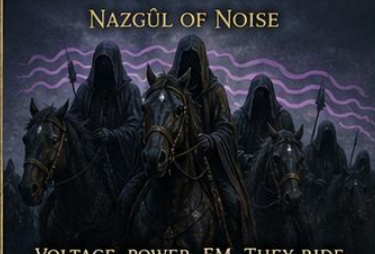
IT WANTS YOUR PRECIOUS KEYS AND IP...

VOICE OF THE CHYPNOSIS



"YOU WILL DO AS I COMMAND. THE BITS WILL REVEAL THEMSELVES."

NAZGÛL OF NOISE



VOLTAGE. POWER. EM. THEY RIDE TO BREAK YOUR DEFENSES.

YOU SHALL NOT PASS... SECURELY



NOT EVEN GANDALF CAN BLOCK A HARDWARE GLITCH.

ONE DOES NOT SIMPLY



ASSUME THE HARDWARE IS INNOCENT.

THE FELLOWSHIP OF SECURE DESIGN



DEFENSE IN DEPTH. TRUST NO LAYER. PROTECT THE JOURNEY.

IN THE LAND OF MORDOR, WHERE THE GLITCHES LIE



DESTROY CHYPNOSIS. THROW IT INTO THE FIRE!



## CHYPNOSIS:

SMALL IN SIZE. EPIC IN CONSEQUENCES.

ONE GLITCH TO RULE THEM ALL. ONE GLITCH TO FIND THEM. ONE GLITCH TO BRING THEM ALL... AND PUT YOUR SECRETS TO SLEEP.

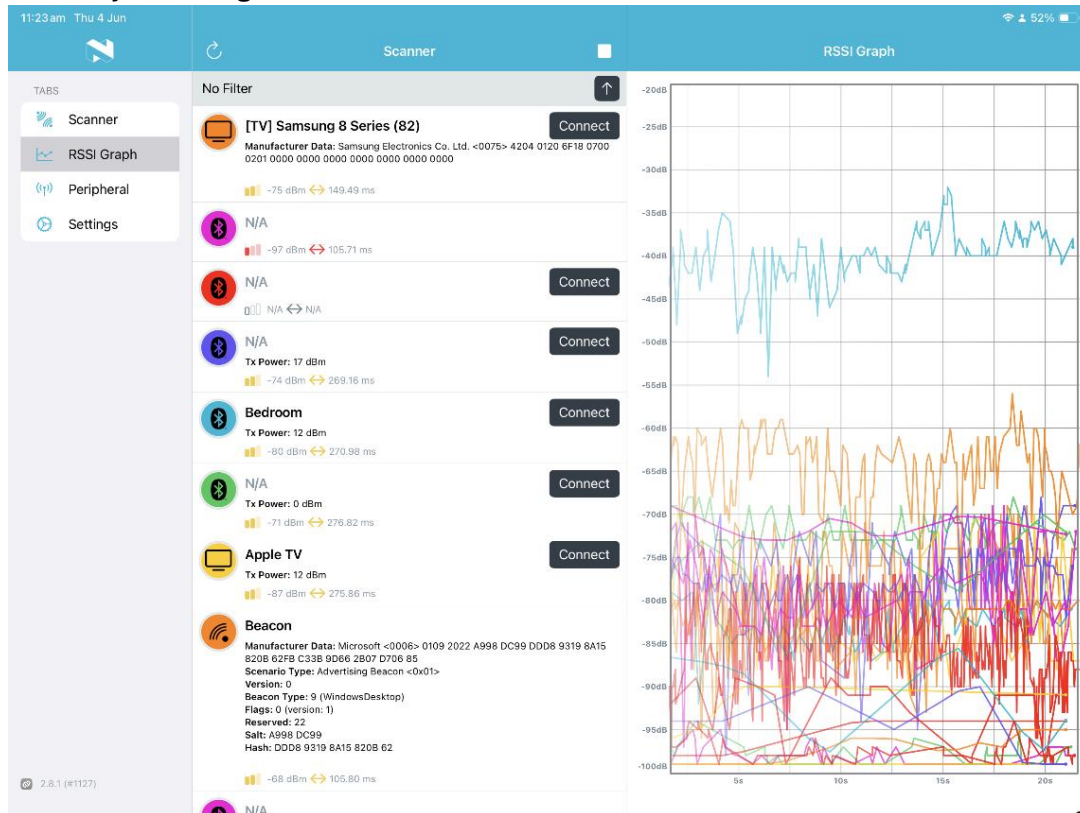




## 2. Security Engineering

### Bluetooth traces

As a bluetooth experiment, I decided to look into how a device could do localisation with Bluetooth signal strength. There has been lots of research using Bluetooth tags to map out the location of a person based on signal strength, channel sounding etc. I wanted to test out how good it is with COT(Commodity Off shelf Technology ) device. To do this experiment quickly and thoroughly I decided to use the Nordic NRF Connect App on the Appstore as I have used it before and Nordic themselves are one of the main manufacturers of BLE chipsets. I like the app because it has an RSSI plot overtime which makes it easier to see trends. With some initial testing this is initially what the plot looked like just remaining static, which as you can see is very busy and confusing and hard to find any meaningful sort of information from.



Therefore, I decided to use the mathematical model of the RSSI value so that I can use a threshold to filter out devices in proximity to me ,this can be shown in the Friis Transmission Equation:

$$\frac{P_r}{P_t} = G_r G_t \left( \frac{\lambda}{4\pi d} \right)^2$$

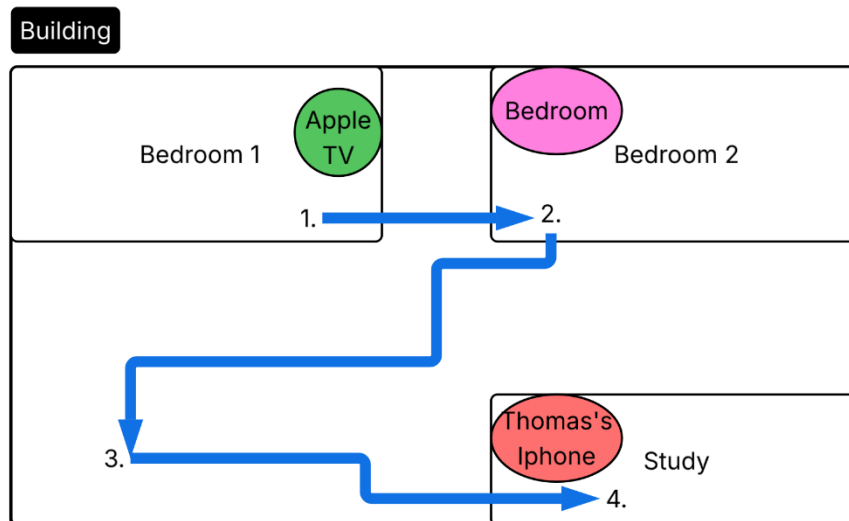
Where  $P_r$  is the received power,  $P_t$  is the transmitted power,  $G_r$  is the received antenna gain,  $G_t$  is the transmitted antenna's gain,  $\lambda$  is the wavelength and  $d$  is the distance between the transmitter and receiver. Now using this equation I make the assumption

that the antenna gain is approximately 1 since most BLE nodes want to have as close to isotropic radiation pattern as they can get, as they cannot assume the placement of the device with respect to a receiver. Therefore, the free space path loss in db can be approximated as:

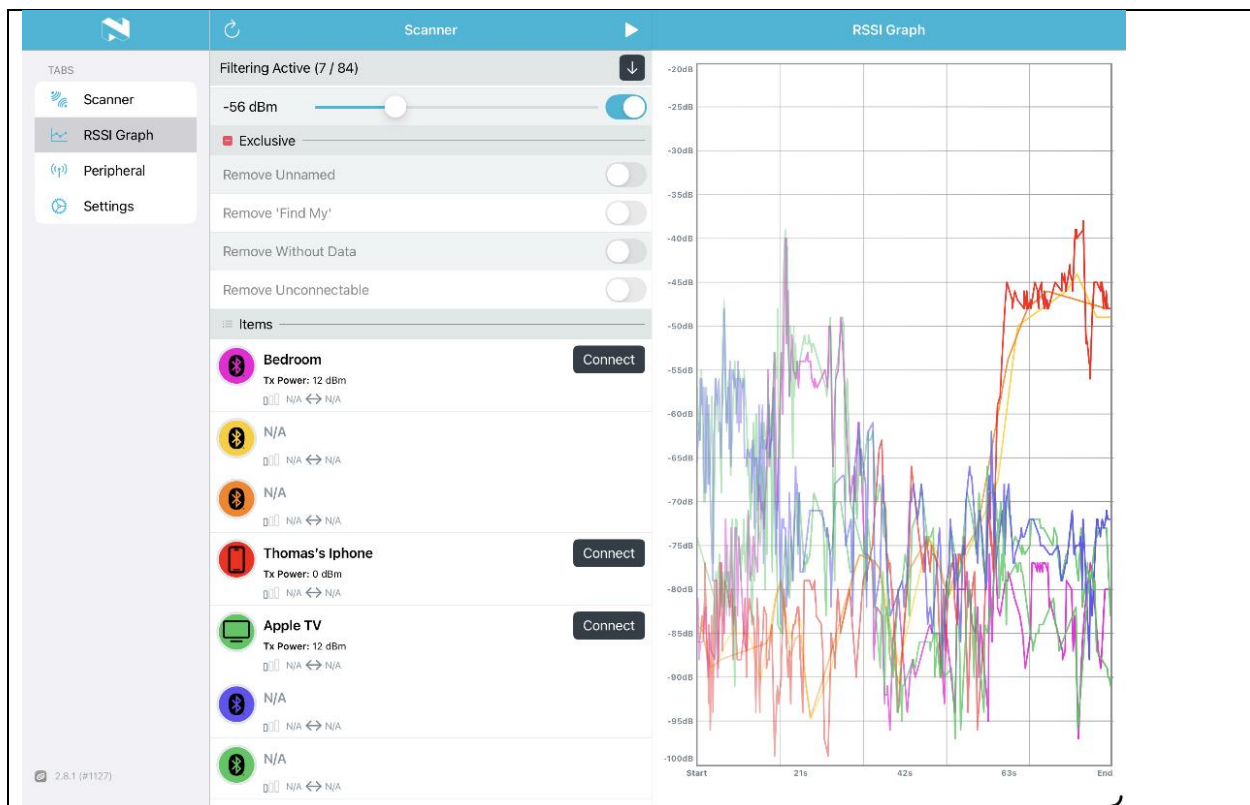
$$FSPL = 20 \log\left(\frac{4\pi d}{\lambda}\right)$$

Now I want to localize myself to a room that I am in, hence I made the assumption of approximately wanting to see device within 2 meters of my location, hence by plugging in the numbers (BLE is 2.4GHz), I found that the path loss should be approximately 46dB, considering the additional losses of radiation efficiency which is generally 30% for low budget devices that we will be considering which is an additional 5dB in loss. In addition to account for further obstacles and things I will add another 4dB in loss. Overall we therefore would expect approximately a 55dB loss to the signal. Furthermore, we assume for most standard devices the transmit power is about 1mW or 0dBm. Therefore, by using the 55dB figure we have a much less chaotic and busy view of devices and can start to localize my movements.

A rough floor plan of my house and device placement is as shown below (without oversharing that much), with my path in blue each stage numbered and the key devices shown to match the respective trace colours on the application.



The results of this experiment, show a clear distinction of my location in each rooms. This can be seen in the RSSI plot on the application. This can be a concerning security vulnerability since these device could track off my device such as a phone or watch to monitor my whereabouts and report it to others. This data could then be processed to determine when I am not home or sleep that either a software attack or physical attack could be started. And may become a reconnaissance technology.



From a different perspective, there is a greater problem with this issue which has been recently addressed. There may be cases when we only want to interact with a user if they are “close enough” to the device (e.g a smart lock) or are the closest. The RSSI signals vary significantly and have a lot of noise, so thresholding may be unreliable, additionally notice the difference in power between the phone and the bedroom devices, this can come down to a few factors such as the transmit power and antenna design. This can allow our assumption of the transmit power to be exploited, and an adversary may increase their power so that our program may believe the device they are using is closer than they actually are and allow them to say unlock a door without being caught on camera. Recently, this was considered a major security risk (imagine a person driving by in their car unlocking everybody’s doors with their maximum tint and sunglasses), the alternative to using the power was what the Bluetooth SIG introduced as channel sounding, which uses the phase and time delay of signals to estimate the distance of a person. Whilst this is not unbreakable it certainly makes it much more difficult for an adversary rather than having an amplifier boost the signal.

## Physical Security

For the physical security activity, I did not have any low risk locks, tamper proof items or a house alarm that can be tampered with so I decided to pursue unlocking an anti theft lock. To first clarify any concern, **I paid for this item at the store and left the tag on it as**

I left, however this **could simulate** an circumstance where somebody may attempt to steal an item (Evidence Below).



**PENDING: BIG W**  
Shopping

-\$15.00

From some fundamental research this lock has two subsystems inside it, a resonator for the gates at doors to detect it and sound an alarm and a pin that is secured by magnetic bearings to lock the device on. My task was to find a way to unlock the device.

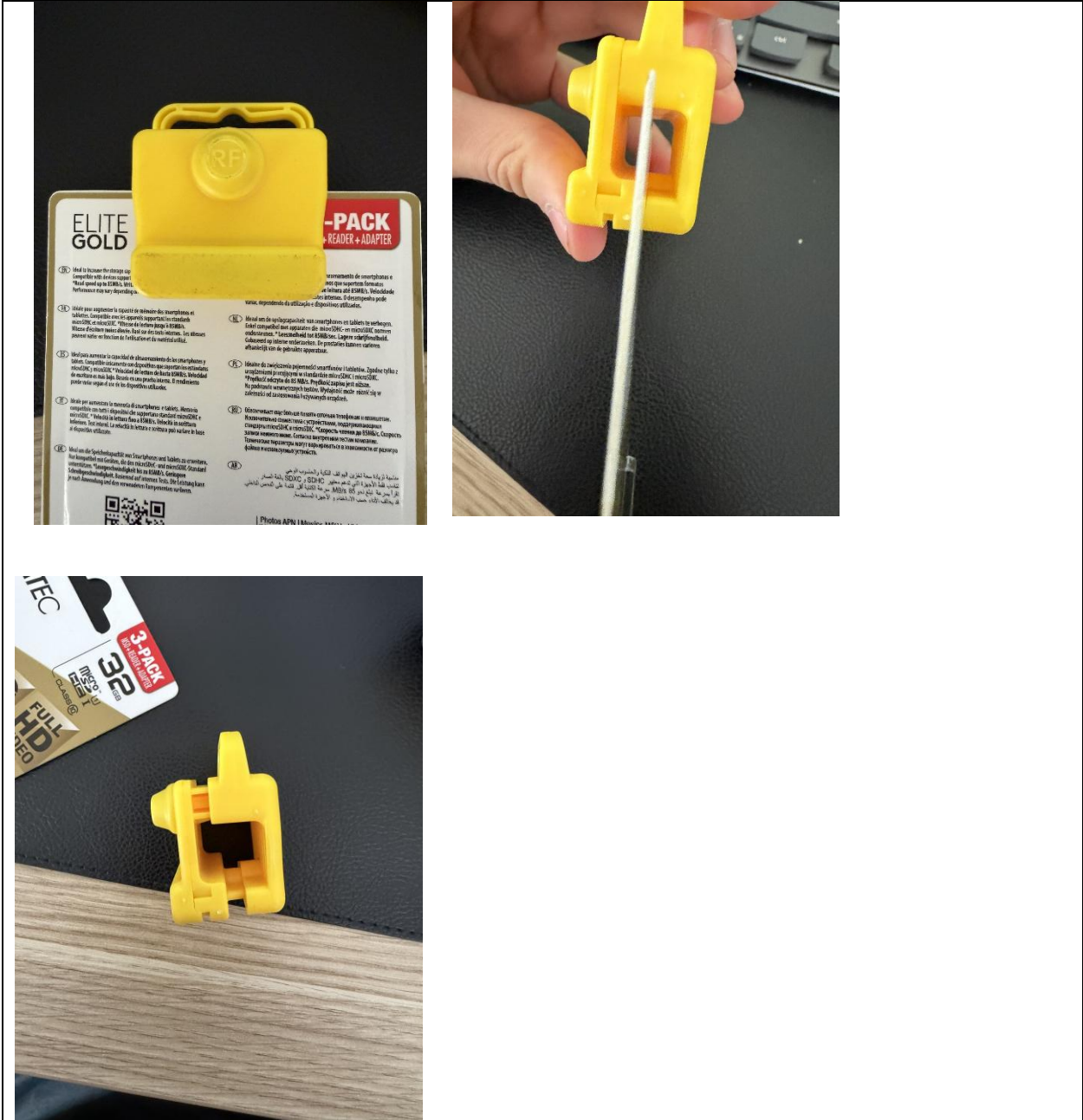
At first to think like an attacker, if I was to find a method of removing these tags, I would want to be able to do it quickly to enable a large scale theft and increase what I have stolen.

A method to remove the lock could be to use brute force and, I could cut it apart, but that would take a while, require large tools and be hard to hide inside a store. Hence, this is a solution but not a good one for a large-scale. A better solution would be to find a way that is quick, robust and cannot be seen visually because stores often also rely heavily video surveillance (Making the assumption that they will be able to see somebody tampering).

Therefore, I thought to replicate the same unlocking device which is at the check out, which contains a strong magnet to shift the bearings holding the pin in place (can conceal magnets in clothing and fields are invisible). Therefore, I used a strong cluster of magnets to tap against the pin surface (tapping was only necessary because my magnets weren't strong enough) and this released pin. The added benefit of this is that if somebody later was to try and distinguish a stolen item from a non-stolen item in investigation, they will not be able to tell since I have imitated the checkout process.







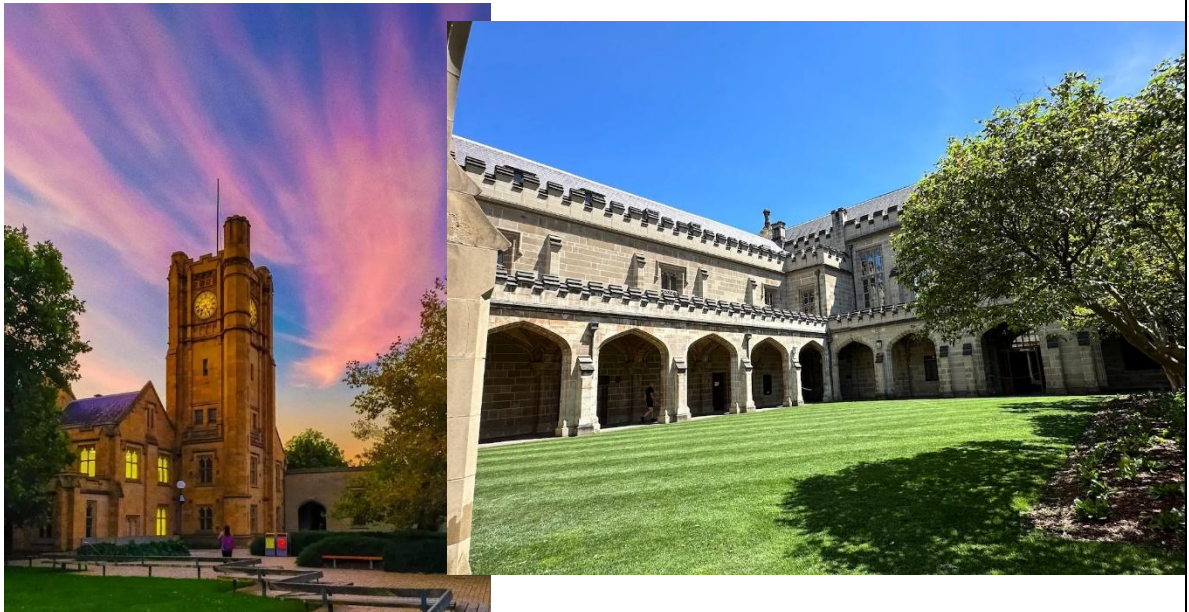


## Photo Detective



- At first by looking at the architectural style it appears as an old historic building from the early 1900s or even earlier. The use of sandstone style suggested to me this is an Australian building.
- After searching, there was no religious icons or tells on the building so I concluded it was not a church and hence must have some other purpose.
- After looking at the light post, lawns and paths this did not look like a regular public historic building, and I began to conclude this must be at a university.
- Due to the age of this building universities such as UNSW and UTS can be ruled out as they are not old enough.
- I began to search through some of the oldest universities in Australia, to understand their ground to see if I could get a hint.

- I began with university of Melbourne but they seemed to have a much less gothic appearance:



- Looking at buildings in university of Sydney the gothic touches seemed to match



- Then I searched “gothic buildings university of Sydney” and found a match of this building from a side view (note the spires on each pillar matching and the window shapes):



- Hence I found that it was MacLaurin Hall at the University of Sydney!

### 3. Extended

#### Week 1 Extension Wargames

During this weeks war games I learnt a lot of different methods of SQL injection. I found that the underlying weakness comes in the nuance of how the user code is added to the SQL command. If a direct string entry is added it gives the user the ability to change the overall command beyond the intention of an argument, this can be exploited to change the security logic of databases and grant access to adversaries.

#### SQLi 1

The SQLi 1 code follows a simple structure: when the button is pressed, a query links the secret with the session ID in the database. The assumptions made in this design are:

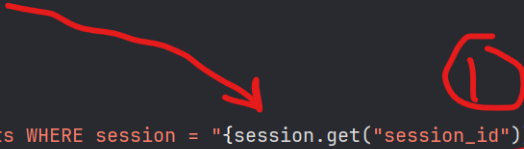
1. Users will only know their own Session IDs
2. Users will only use normal alphanumeric strings as session IDs
3. Each Secret is linked to one session ID

However, these assumptions are clearly false on multiple levels and can be exploited. As shown below, the design assumes the session ID is just an alphanumeric string that can be inserted directly into the query.

```

56 def get_secrets(session_id=None):
57     # if they supply a session_id, use it for their session (we trust them <3)
58     if session_id:
59         session['session_id'] = session_id
60     elif not session.get('session_id'):
61         return []
62
63     results = execute(f'SELECT * FROM secrets WHERE session = "{session.get("session_id")}"')
64     return results.fetchall() if (results) else []

```



A critical flaw appears if the session ID starts with a quotation mark, because this closes the session string being used. That lets the user append any SQL query or condition they want to the query.

Now we want to find this user's secrets. Since we do not have their exact session ID, we cannot pull only their entry directly, so it is easier to collect every secret and identify theirs from the full result. We can do this by forcing the query to select every entry using a condition that is always true. A simple way is to OR the previous condition with one that always evaluates true, such as "1" = "1". By combining this with the flaw above, a possible injection vector is:

“ OR “1”=“1”

Secret Diary Home

Your session ID

Get your secrets!

Your secret

Add a new secret!

| Your secrets                |
|-----------------------------|
| COMP6841{FLAG-SQL1_15_EAZY} |
| None                        |
| sdf                         |
| None                        |
| cc                          |
| None                        |
|                             |
| None                        |
| None                        |
|                             |
|                             |
| COMP3311                    |

## SQLi 2

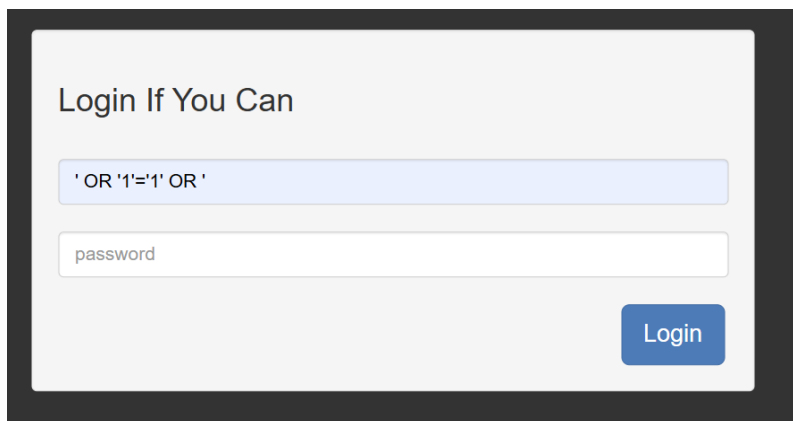
From an overview of the code used for this login page, similar assumptions are made, but only for the username and not the password. This can be seen below.



The username entry will be swapped in, whereas the password is added as an argument. While both are used in the query, the handling is very different. The username is inserted directly into the query, similar to SQLi 1, whereas the password is passed as an argument to the function and does not alter the query itself.

Now to put the attacker mindset on, *we want to login without logging in*. We have two ways to get through: directly or around it. Breaking the validation itself with another password or brute force is slow and annoying, so it is easier to go around it. We can edit the query so the password is ignored and the username is cancelled out by a permanently true statement. So we place a ' and then OR '1' = '1'. For the password section, we can OR with it to remove its effect. Hence, the input vector to disrupt the section will be:

' OR '1'='1' OR '



Welcome admin

COMP6841{HMM\_I\_DONT\_RECOGNISE\_YOU}

[Logout](#)

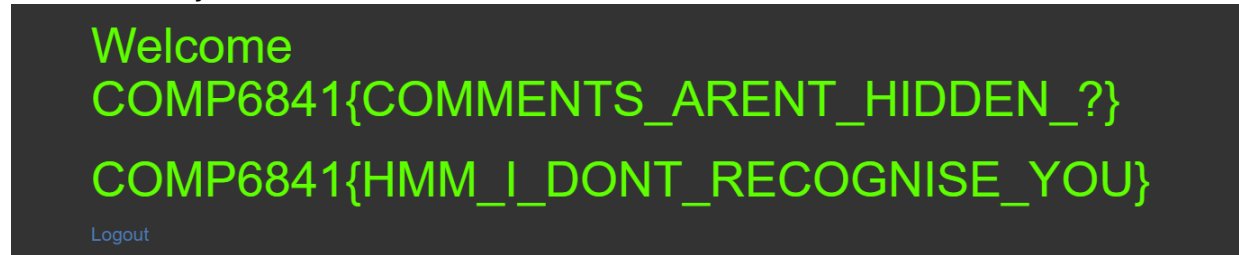
## SQLi 2.5

The code provided here is basically the same as SQLi 2.5. Now we want to find the other flag. Normally I would be a bit lost on where to look apart from guessing, but there were some helpful hints in the **super cool user comments**.

```
61 def login(username, password):
62     # we leave commented code in production because we're cool.
63     # SELECT flag FROM my_secret_table
64     results = execute(f"SELECT username FROM users WHERE username = '{username}' and password = %s", (password,))
65     return results.fetchone() if (results) else False
```

That made me suspicious that there might be an isolated database containing the flag, based on the naming and exact SQL syntax shown. I used a similar injection method as

above, but had to be careful with the syntax. By using the UNION operator, I could combine the normal retrieved entries with those from the secret table. To keep the SQL valid, I commented out the password section with --. This gave a valid SQL injection to access the my\_secret\_table.

A screenshot of a login form titled "Login If You Can". The form has a light grey background. It contains two input fields. The top input field is highlighted with a blue border and contains the text "' UNION SELECT flag FROM my\_secret\_table --". The bottom input field has a light grey border and contains the placeholder text "password". To the right of the password field is a blue button with the text "Login" in white.

### SQLi 3

From a brief look at the code, it seems the developers tried to use a blacklist to block injected operators from reaching the server. Their assumption is that we will type injected code neatly in lowercase or uppercase. That is the critical flaw, because we can type in aLtErNaTiNg CaSe and the query will still pass the blacklist and remain valid, since SQL is not case-sensitive. Also, any other missed operator or case combination would get through, which shows why blacklists are weak: it only takes one missed combination to fail. So we can do exactly that and introduce a UnIOn from the top\_secret table with a comment to remove the remaining conditions. Hence, the flag was able to be captured:

```

13 @app.route('/', methods=['GET'])
14 def root():
15     query = request.args.get('q')
16     if query is not None:
17         # super powerful WAF
18         for bad in ['union', 'select', 'from', 'where', 'UNION', 'SELECT', 'FROM', 'WHERE']:
19             query = query.replace(bad, '')
20
21         return render_template('home.html', query=query, results=get_results(query))
22
23     return render_template('home.html')

```

```

if q:
    results = execute(f"SELECT * FROM search_engine WHERE title LIKE '{q}%' OR description LIKE '{q}%' OR link LIKE '{q}%'")
else:
    results = execute("SELECT * FROM search_engine")
return results.fetchall() if (results) else []

```



Number of results for **UNioN SEIEction topsecret, NULL, NULL FRom secrets -- '**

**Twitter**  
[https://twitter.com/hpy\\_downunder](https://twitter.com/hpy_downunder)  
 Totally not a social media influencer...

**Hackerone**  
<https://hackerone.com/acoupleofhacks>  
 HackerOne is where I sometimes hack...

**Bugcrowd**  
<https://bugcrowd.com/hpy>  
 I hack on here a little more often

**COMP6841{FLAG-SQL1\_15\_N0T\_50\_EA2Y}**  
 None  
 None

## Analysis and Reflection

### Tutorial 1: Orientation

During the tutorial, there was a significant delay because the tutor did not arrive until midway through the session. Despite this, we were still able to discuss both the initial rumble scenario and the later scenario involving false reporting.

## **Initial Rumble**

- Many people initially relied on their peers' reactions as evidence that an earthquake was happening.
- I found this response questionable because the evidence was weak: it was based mainly on brief observations and other people's opinions. In cybersecurity and crisis situations, human judgement can often become the weakest link.
- It was interesting to see how classmates reacted when I suggested we should challenge the earthquake assumption. That moment made me more aware of how social engineering can exploit our default behaviours and lead us into errors or traps.
- My own response would be to seek additional observation or verification before acting. I would want stronger evidence that hiding was the right course of action, rather than immediately ruling out alternatives such as evacuating or helping someone who was injured. Since the only signs were noise and shaking dust, I would not have been confident that it was actually an earthquake, especially in Australia.

## **Fake News and No Power Scenario**

- When the news update and the "stay calm" scenario were introduced, most people still believed the safest response was to shelter under a table or in a doorway.
- However, the vague information did not seem to make people feel more confident or secure in their decision. This was an interesting reminder that, in a crisis, simplified or overly technical explanations can reduce trust if the audience does not fully understand them. Even when the advice may be correct, poor communication can make people feel uncertain and unsafe, due to our inherent fear of the unknown or ambiguous.



## 4. Project

### Select your Project

At this stage I have a few areas of research and found the following ideas to be interesting, but am happy to receive feedback for different ideas, I work and tend to work mostly on embedded systems and IoT devices so I was considering one of the following:

- Investigating the attack surfaces and vulnerabilities in BLE Auracast Networks(already have hardware)
  - Develop a small Auracast Application, look at common mistakes or vulnerabilities and impacts of such vulnerabilities with a demonstration due to misconfiguration and fault injection.
  - Report back critical configuration steps and vulnerabilities present on such platforms.
- Develop a automotive radar spoofing device which can make my car think that there is a large vehicle moving in a place where their isn't. (i.e could trigger emergency braking and exploit control over the vehicle) also could develop meta surface materials to make a surface appear "invisible". Highlighting the vulnerability and mixing of control and data planes in autonomous systems
- **Practical Spoofing/CEMA CTF:** This project would explore multiple challenge levels built around receivers that make decisions based on incoming data, helping users understand how vulnerabilities can be exploited at the hardware and systems level. The aim is to create a practical, low-level learning platform for studying both attack techniques and defensive design in autonomous and communication systems. One motivation for this idea is the lack of accessible training platforms in counter-autonomous and electronic attack scenarios. Because agencies rarely disclose the algorithms used in real systems, it is difficult for beginners to gain hands-on experience without expensive equipment, legal restrictions, or unsafe experimentation. A controlled CTF-style environment could provide a practical and legitimate way to learn how real-world vulnerabilities are identified, exploited, and mitigated.
  - **Level 1: FM Radio:** A transport authority wants a way to broadcast emergency information inside a tunnel without requiring drivers to manually change stations. The challenge is to determine how a radio broadcast could be transmitted from a .wav file in an environment where there is no time to coordinate with other radio operators.
    - Users are provided live Spectrum Analysis View, Frequency Bands, Audio, Python Input Terminal for RF-specific controls, and a configurable RF Frontend with user input.

- **Level 1.25: FM Radio, but Protected:** After deployment, the authority realises that FM radio is publicly accessible and wants the system updated so that only authorised broadcasts can trigger this behaviour.
  - Users are provided a live spectrum view of the radios, spectrogram view, Python interface for RF/analogue settings and control of the RF frontend and stack, and a C interface for the embedded control logic with a .wav opener already provided.
  - Users are provided a sample of how adversaries have taken over in the past.
- **Level 1.5: FM Radio, Multiple Messages:** The client now wants to extend the deployed system so it can also send UART messages to roadside billboards, using the existing hardware and working within tight budget constraints, without drivers noticing.
  - Users are provided a live spectrum view of the radios, spectrogram view, Python interface for RF/analogue settings and control of the RF frontend and stack, and a C interface for the embedded control logic with a .wav opener already provided.
  - Audio output that their signal creates in a car.
  - Printout on boards.
- **Level 1.75: Radio Jammers Are Nasty:** The authority becomes concerned that an attacker could exploit outdated IDs, keys, frequencies, or baud rates to take control of signs and audio systems. This stage focuses on evaluating and attempting to break the design in a controlled simulated CTF environment. This also simulates a more coordinated attack surface.
  - SQLi-style local challenge interface to find a key-related flag.
  - Users are provided a live spectrum view of the radios, spectrogram view, Python interface for RF/analogue settings and control of the RF frontend and stack, and a C interface for the embedded control logic with a .wav opener already provided.
  - Audio output that their signal creates in a car.
  - Printout on boards.
- **Level 1.9: Roadrunner and Coyote:** A second agency challenges the robustness of the earlier design and asks for a security assessment of its own audio communication system used to direct freight services to park at specific destinations.
  - Users are provided a live spectrum view of the radios, spectrogram view, Python interface for RF/analogue settings and control of the RF frontend and stack, and a C interface for the embedded control logic with a .wav opener already provided.
  - Audio output that their signal creates in a car.
  - Printout on signs or lights.
- **Level 2: My RC Plane Has Lost Contact :** A company using RC aircraft for bushfire analysis is concerned about signal loss in rapidly changing radio conditions. If contact is lost, the aircraft reverts to GPS-based return-to-

home behaviour, which may no longer be safe (airfields might catch fire or be repurposed). The challenge would be to design software or transmission logic that enables controlled recovery of the aircraft without relying on the original receiver, such that it can land on a different platform safely.

- Users are provided a live spectrum view of the radios, spectrogram view, Python interface for RF/analogue settings and control of the RF frontend and stack, and a C interface for the embedded control logic with an example data sender already provided.
- An interactive map of the RC plane location is shown to construct a simulation.
- **Level 2.5: GPS Spoofing with Sensor Fusion Protection** :The aircraft manufacturer adds IMU-based validation to check whether GPS data is physically plausible before the aircraft trusts it.
  - Users are provided GPS traces, IMU telemetry, residual plots, navigation filter outputs, an interactive flight map, and selected C/C++ source code for the embedded validation logic.
  - Users can generate modified GPS-like data in Python and test whether the embedded navigation logic accepts or rejects it.
  - The focus is on sensor fusion, residual checking, plausibility constraints, spoof detection, and the difficulty of influencing a system that cross-checks multiple data sources.
- **Level 2.75: Distributed Sensor Trust** : The aircraft is integrated into a wider bushfire monitoring network using LoRa-style remote environmental sensors to detect where the fire is.
  - Users are provided a network topology view, simulated LoRa packet monitor, RSSI/SNR display, sensor telemetry dashboard, trust-score output, Python packet/channel generator, and C-based embedded packet parser.
  - Users can observe how sensor reports influence the aircraft, gateway, and control-centre decision logic.
  - The focus is on distributed trust, node identity, packet integrity, stale keys, sensor plausibility, and how false or inconsistent information can affect operational decisions.
- **Level 3: My Smart Radio Is So Cool, AI-Protected Cognitive Radio** : A state emergency service deploys an AI-managed cognitive radio network to coordinate autonomous bushfire response assets, including UAVs, ground sensors, mobile command vehicles, and radio relays. The vendor claims that the onboard AI can detect spoofing, communication attacks, sensor manipulation, and abnormal RF behaviour before they affect the mission, but many of their clients are sceptics and want it rigorously tested.
  - Users are provided a live spectrum view of the radios, spectrogram view, Python interface for RF/analogue settings and control of the

RF frontend and stack, C/C++ embedded control logic, an example data sender, output logs, dataset constructor, and a machine-learning/data-modelling interface.

- Users can observe AI confidence scores, anomaly alerts, channel decisions, packet behaviour, trust scores, communication availability, and mission status.
  - The CTF objective is to test the vendor's claim by influencing the adaptive system's decision-making while remaining below the onboard AI's detection threshold.
  - Example objectives include causing the system to trust a faulty sensor cluster, switch away from the best communication channel, reject a legitimate relay node, or redirect an autonomous asset to the wrong simulated location.
  - The focus is on black-box testing, adversarial inputs, adaptive communications, ML-assisted detection, trust modelling, and CEMA-style decision manipulation.
- **Level 3.1: Model Reconnaissance** - Users observe the adaptive system over time and attempt to determine what features influence the onboard model.
    - Users are provided RF logs, packet timing data, spectrum observations, telemetry outputs, AI confidence values, and historical decision traces.
    - The goal is to infer which features the model appears to trust, such as RSSI, SNR, packet loss, timing variance, node identity, GPS consistency, imu sensor data, windspeed data from external notes etc.
  - **Level 3.2: Adversarial Influence** - Users attempt to craft inputs that influence the system while still appearing normal to the AI-assisted defender.
    - Users can generate RF/channel conditions in Python and interact with the embedded protocol logic in C/C++.
    - The goal is to cause a specific system behaviour, such as changing channel, rejecting a trusted node, accepting a false sensor report, or altering a route decision, while keeping the anomaly score below threshold.
  - **Level 3.3: Capture the Adaptive System** - The final challenge combines RF observations, packet behaviour, sensor data, GPS/IMU validation, and the AI-protected cognitive radio into one CTF scenario.
    - Users must achieve a mission objective while maintaining plausible RF metrics, valid protocol behaviour, believable sensor data, and low AI suspicion.
    - Success is scored using mission impact, detection probability, communication availability, and trust score.

## 5. Wrap up

### Submission Instructions

To submit a valid portfolio entry, complete the activities, save your work as a PDF, and upload it to the relevant week submission page on Moodle.

Acknowledgement: Please note that artificial intelligence has been used for simple grammatical editing my written work.

[1] OpenAI. 2026, *ChatGPT* [GPT5.5], Retrieved June 4, 2026, from <https://chat.openai.com/chat>.